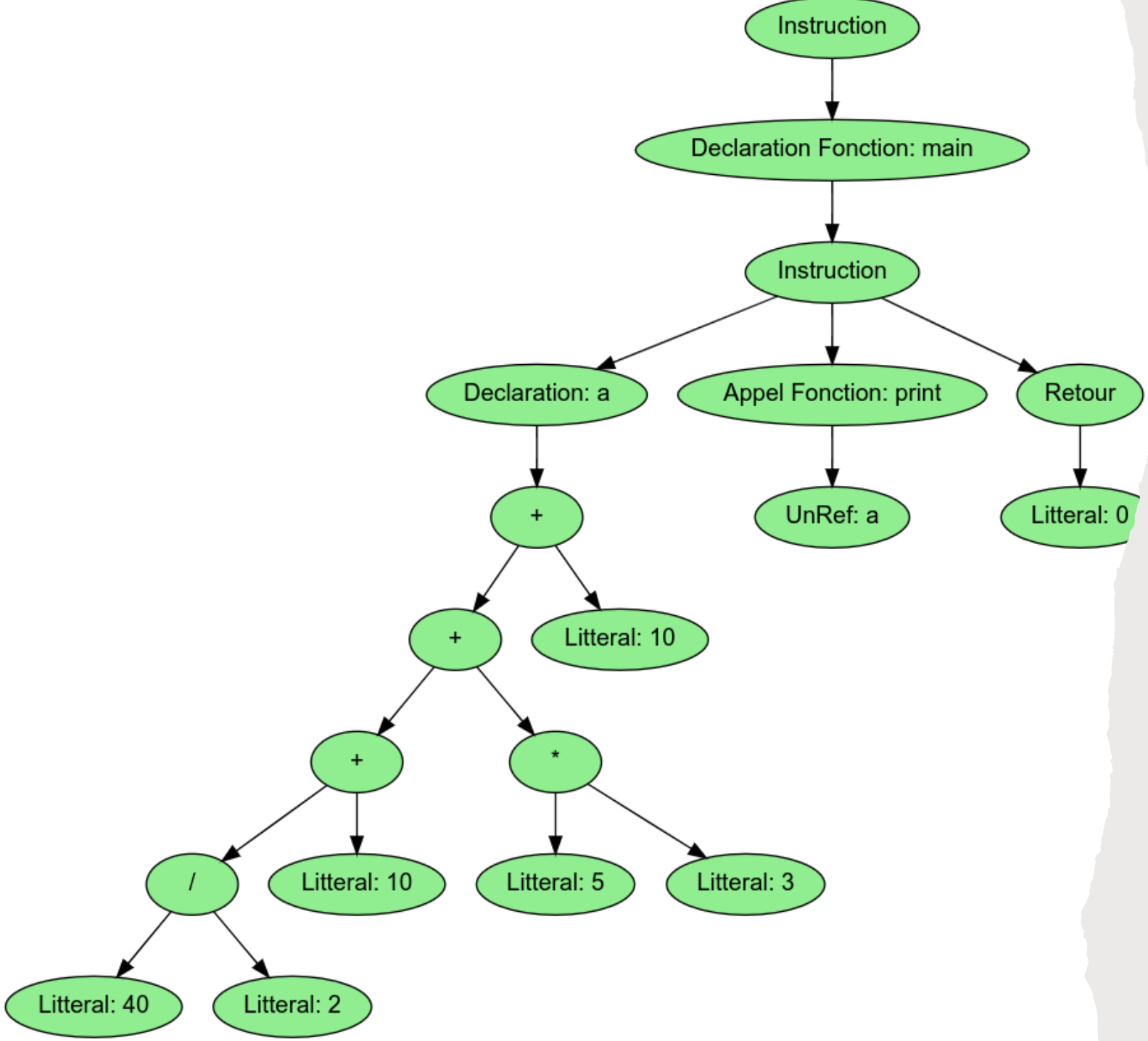

Vue d'ensemble sur le projet

Sprint 1 (Terminé)

- Lire des fichiers sources Prysma et les traiter comme du texte Lire des fichiers sources Prysma et les traiter comme du texte Lire des fichiers sources Prysma et les traiter comme du texte
- Supporter les types entiers (int), flottants (float), booléens (bool)
- Compiler les opérations arithmétiques de base (+, -, *, /) vers du code LLVM
- Permettre au développeur de déclarer des variables et les utiliser dans des expressions
- Valider la syntaxe du code et rejeter les structures invalides via la construction de l'AST
- Supporter les fonctions récursives et un type de retour.
- Signaler les erreurs de syntaxe précisément (numéro de ligne, type d'erreur) plutôt que planter silencieusement
- Ajouter la coloration syntaxique du langage Prysma dans Visual Studio Code

Sprint 2 (Terminé - Focus actuel)

- Exporter l'AST en format texte et graphique (Graphviz) pour faciliter le débogage
- Refonte complète de la gestion mémoire pour une performance maximale
- supporter les opérantes && || == != %
- Supporter l'instruction if,
- Supporter l'instruction while, passer par la pile avec alloca
- Supporter les tableaux programmés en LLVM IR
- Supporter le type string
- Supporter les includes, en utilisant les techniques du langage de programmation C #include "racine/fichier.h"
- Mettre en place un système de compilation multi-thread performant

Sprint 3 (À venir)

- Supporter l'approche orientée objet avec des classes, polymorphisme fait avec une vtable.
- Le Layout de Données (La Structure)
- Les Contrôles d'Accès (Visibilité)
- l'Encapsulation et le Pointeur this
- Le Cycle de Vie (Constructeurs & Destructeurs)
- L' Héritage (La Hiérarchie de Classes)
- Le Polymorphisme (La VTable)
- Analyse syntaxique abstraite pour la logique du langage Prysma, ce qui évitera les problèmes de logique sémantique exemple : declare int a = "string";
- Écrire une bibliothèque de structures de données (listes chaînées, arbres binaires, piles, files)

Choix technologiques effectués

Gestion de la mémoire (LLVM BumpPtrAllocator) : J'ai complètement abandonné les std::shared_ptr pour les nœuds de mon AST. À la place, je gère la mémoire manuellement avec un système d'arena allocator (BumpPtrAllocator) offert par LLVM. C'est beaucoup plus performant : le risque de fuite est limité car l'arena détruit tout d'un coup à la fin, la mémoire est contiguë, ce qui utilise parfaitement la mémoire cache L1/L2 du processeur et réduit la fragmentation.

Système variadique C pour l'affichage : Pour l'affichage (print), j'ai utilisé un système générique variadique natif du C avec un switch case sur les types. C'est totalement invisible pour l'utilisateur, il ne fait qu'appeler print() et le compilateur gère le cast dynamiquement sous le capot.

Ce que j'ai choisi de développer durant l’itération

Tableaux et Strings : Implémentation complète des tableaux (bool, float, int). Je gère maintenant la taille dynamique, la lecture d'un index précis (tableau[2]) et l'affectation (tableau[1] = 20). J'ai aussi ajouté le support natif des chaînes de caractères.

Logique de contrôle : La boucle while et les conditions if/else sont 100% opérationnelles avec la gestion complète des opérateurs logiques et de la négation booléenne.

Multi-threading et Includes : J'ai programmé un orchestrateur capable de gérer l'inclusion de fichiers multi-niveaux. Le tout compile désormais en multi-thread, offrant un gain de performance massif.

Tests automatisés robustes : J'ai mis en place une intégration continue avec installation automatique de LLVM 18, et j'ai fait des tests de charge massifs (jusqu'à 2000 fichiers compilés simultanément) pour garantir la stabilité de l'orchestrateur.

Mes bons coups

Le gain de performance extrême : En combinant l'Arena Allocator (qui évite les allocations dynamiques coûteuses des shared_ptr) et mon nouveau système d'orchestration multi-thread, la compilation est devenue incroyablement rapide, même sur des tests de charge avec 2000 fichiers.

Défis rencontrés

- **Le cauchemar du Multi-thread avec LLVM :** LLVM n'est pas "thread-safe". Des éléments comme llvm::Value ou llvm::AllocaInst causent des conflits majeurs s'ils sont manipulés par plusieurs threads en même temps dans un contexte commun. J'ai dû concevoir une classe d'orchestration séparée en deux passes distinctes et utiliser un polymorphisme statique pour avoir de vrais/faux mutex. Cela m'a permis de découpler les registres de variables/fonctions et d'éviter les crashs.
- **La récursivité infinie des appels :** En voulant permettre d'insérer un appel de fonction directement dans une addition mathématique, le parseur plantait parce qu'il ne voyait que du texte qu'il ne comprenait pas. J'ai dû abandonner mon système d'extraction d'équation complexe pour mettre en place un système de "récursivité mutuelle" entre le constructeur de l'arbre d'équations et celui des instructions. C'était difficile à conceptualiser, mais ça a réglé le problème élégamment sans avoir à scanner manuellement les parenthèses.

Ce qui reste à développer pour la prochaine itération

L'itération 2 a rendu le langage Prysma vraiment utilisable (multi fichier, boucles, tableaux, multi-threading), mais il manque la brique finale pour les gros projets : la Programmation Orientée Objet.

Je vais devoir implémenter le système de classes et le polymorphisme en utilisant une "vtable" (table de méthodes virtuelles) générée en LLVM IR.

Je dois aussi écrire la bibliothèque standard de Prysma (Listes chaînées, Arbres binaires) pour prouver que mon compilateur est capable de compiler des structures de données complexes par lui-même.